CATune: Structural Constraint-Aware Bayesian Optimization for DBMS Configuration Tuning

Fangping Lan
Temple University
Philadelphia, PA
fangping.lan@temple.edu

Qi Zhang
Temple University
Philadelphia, PA
qi.zhang@temple.edu

Eduard Dragut
Temple University
Philadelphia, PA
edragut@temple.edu

ABSTRACT

Modern DBMSs expose hundreds of configuration knobs, resulting in a high-dimensional and heterogeneous search space that makes automated tuning costly. Existing ML-based tuning systems typically treat the configuration domain as box-constrained and rely on workload feedback to implicitly capture inter-knob relationships. However, DBMS documentation specifies deterministic knob dependency constraints—particularly ordering constraints—that characterize structurally valid regions of the configuration space. We present CATune, a constraint-aware Bayesian optimization (BO) framework that models deterministic inter-knob ordering constraints as structural components of the search domain. Instead of learning feasibility boundaries through sampled violations, CATune performs optimization within a constraint-consistent subspace. We develop a topology-aware sampling strategy that respects dependency structure during exploration and avoids the inefficiencies of post-hoc constraint handling To enable automated constraint discovery, we further design a precision-first extraction pipeline that combines LLM-based parsing with reliability safeguards to mitigate hallucinated dependencies. Experiments on PostgreSQL using TPC-C and TPC-H workloads show that CATune substantially improves both sample efficiency and final tuning quality across surrogate models and BO frameworks. Under default ranges, CATune reaches the baseline optimum up to 12.5× faster and improves throughput by up to 63.37%. The improvements persist under knowledge-guided reduced ranges and alternative optimization implementations. These results demonstrate that explicitly modeling system-defined deterministic ordering constraints enhances optimization robustness and system stability.

PVLDB Reference Format:
Fangping Lan, Qi Zhang, and Eduard Dragut. CATune: Structural Constraint-Aware Bayesian Optimization for DBMS Configuration Tuning. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lanfangping/CATune.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Modern Database Management Systems (DBMSs) expose hundreds of configuration knobs that govern memory allocation, concurrency control, logging behavior, and query execution strategies [34]. While this flexibility enables performance optimization across diverse workloads, it creates a high-dimensional and heterogeneous configuration space that is difficult to navigate [48]. For example, PostgreSQL v13 provides 263 knobs spanning continuous, integer, and categorical domains. In cloud environments, where workloads and hardware configurations vary widely, manual tuning by database administrators (DBAs) becomes increasingly infeasible [54].

To address this challenge, prior work has proposed automated and ML-based tuning methods. Existing approaches broadly fall into two categories: (1) offline transfer-based methods that pre-train models on benchmark workloads and transfer knowledge to new deployments [47, 53, 58], and (2) online search-based methods that iteratively select configurations using search algorithms, reinforcement learning (RL) [5, 28, 44, 54], or Bayesian optimization (BO) techniques such as SMAC [12, 22, 27, 30, 47, 56, 57].

Although these methods can eventually identify high-performing configurations, they incur significant tuning cost [20, 22]. State-of-the-art systems often require *hundreds of iterations* to stabilize performance [55], and each iteration may execute expensive workloads. This cost arises from two challenges: the large number of knobs requiring exploration and the broad vendor-provided ranges associated with each knob. These ranges are intentionally conservative to ensure generality but substantially enlarge the search space and may include unstable or invalid regions [35, 48].

Meanwhile, human DBAs rarely rely on blind trial-and-error. Instead, they use domain knowledge documented in manuals and community discussions. Recent knowledge-aware tuners leverage Large Language Models (LLMs) to extract suggested values or reduce search regions [24–26, 44, 45]. However, these approaches primarily focus on identifying promising value ranges.

An under-exploited form of knowledge lies in system-defined deterministic knob dependency constraints, i.e., relationships among knobs that define structurally invalid or suboptimal regions (e.g., ordering constraints such as $A \leq B$). These dependencies are documented in manuals and forums (Table 1); to our knowledge, they are not explicitly encoded in existing ML-based tuning systems. As a result, current tuners treat the configuration space as box-constrained and implicitly learn such relationships through trial-and-error feedback, leading to wasted evaluations of invalid configurations. In contrast, we explicitly encode deterministic inter-knob ordering constraints as structural components of the search domain.

We hypothesize that explicitly encoding deterministic knob dependency constraints into the optimization process can substantially

Table 1: Three examples of knob dependency constraints found in PostgreSQL V13 Documentation

Knob	`superuser_reserved_connections`
Description	Determines the number of connection "slots" that are reserved for connections by PostgreSQL superusers ... The value must be less than `max_connections`...
Constraint	`superuser_reserved_connections` < `max_connections`
Knob	`max_parallel_workers`
Description	... Also, note that a setting for this value which is higher than `max_worker_processes` will have no effect, since parallel workers are taken from the pool of worker processes established by that setting.
Constraint	`max_parallel_workers` ≤ `max_worker_processes`
Knob	`max_parallel_workers_per_gather`
Description	Sets the maximum number of workers that can be started by a single Gather or Gather Merge node. Parallel workers are taken from the pool of processes established by `max_worker_processes`, limited by `max_parallel_workers`...
Constraint	`max_parallel_workers_per_gather` < `max_worker_processes`, `max_parallel_workers_per_gather` < `max_parallel_workers`

improve sample efficiency and robustness. By filtering structurally invalid regions prior to evaluation, constraint-aware tuning reduces unnecessary trials and accelerates convergence.

We present CATune, a system that encodes ordering constraints directly into the configuration domain and integrates them with BO-based optimizers. Ordering constraints naturally induce a structured dependency graph, which we model as a *topology graph*. Unlike classical constrained BO methods [13, 14, 36, 40, 56], which model constraints probabilistically and learn feasibility boundaries by sampling infeasible configurations, CATune restricts optimization to a constraint-consistent subspace prior to search. Structural constraints are enforced in both exploitation (Section 5.1) and exploration (Section 5.2).

However, simple constraint-handling strategies typically enforce dependencies after candidate generation, either by rejecting infeasible configurations [17] or repairing them to satisfy the constraints. Under multiple dependency constraints, these post-hoc approaches can incur substantial overhead due to excessive rejected samples or additional repair operations. To address this, we design a topology-aware sampling strategy that respects the dependency graph during sample generation, ensuring feasibility by construction while maintaining high sampling efficiency (Section 5.2.3).

Extensive experiments demonstrate that CATune improves both final tuning quality and convergence speed across workloads, surrogate models, BO implementations, and search-space regimes (Section 8). Compared to penalty-based strategies, proactive constraint filtering yields faster convergence by eliminating infeasible regions prior to optimization. Ablation studies confirm the effectiveness of topology-aware sampling (Section 9.2). Beyond performance gains, we analyze the system-level impact of constraint enforcement (Section 9.1). Correct ordering constraints prevent DBMS startup failures and improve reliability, whereas hallucinated constraints can negatively affect optimization performance.

Although LLMs demonstrate strong capability in extracting tuning knowledge from textual sources [50, 59], they are not immune to hallucination [1, 52]. Our empirical analysis (Section **??**) shows that among 30 extracted constraints, 6 were hallucinated. Notably, two of these hallucinated constraints led to measurable degradation in tuning efficiency. These findings highlight the risk of incorporating automatically generated constraints. To mitigate this risk, we design a precision-first constraint extraction pipeline that prioritizes correctness when leveraging LLM-based agents for automated dependency discovery.

Contributions. Our contributions are:

(1) We propose CATune, a DBMS tuning system that structurally encodes deterministic knob dependency constraints to restrict optimization to a constraint-consistent subspace.
(2) We study the sampling efficiency of three simpler constraint-handling strategies and observe that post-hoc configuration modifications can incur substantial overhead.
(3) We introduce a topology-aware sampling strategy that avoids low acceptance rates inherent in rejection-based constraint handling.
(4) We design a precision-first LLM-assisted extraction pipeline that automatically discovers and refines knob dependency constraints while mitigating hallucination.
(5) We conduct extensive experiments demonstrating improved convergence speed and final performance across workloads, surrogate models, and BO frameworks.
(6) We provide a systematic analysis of constraint employment strategies, hallucinated constraints, and the system-level impact of constraint enforcement.

2 MOTIVATION

This section motivates constraint-aware tuning by showing that existing ML-based systems waste budget on invalid configurations despite documented structural dependencies, and that current frameworks lack a systematic mechanism for enforcing such dependencies during optimization.

M1: Tuning budget waste significantly increases the cost of ML-based DBMS tuning. State-of-the-art ML-based tuning methods typically require hundreds of iterations to converge to a high-quality DBMS configuration, resulting in substantial tuning overhead. However, a non-trivial portion of this budget is effectively wasted. In practice, many sampled configurations are either invalid or uninformative. For example, in our preliminary experiments, which was conducted on PostgreSQL using the TPC-C workload with 46 knobs and documentation-specified search ranges, approximately 35% of tuning iterations led to DBMS crashes. Additionally, numerous configurations violate knob dependency constraints (e.g., `superuser_reserved_connections` must be strictly smaller than `max_connections`), producing executions that are rejected or semantically meaningless. Such wasted iterations not only increase

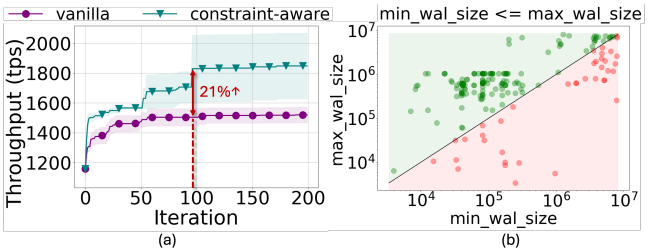

Figure 1: (a). The preliminary tuning performance improvement on the TPC-C workload using 10 knobs with/without constraint-aware tuning; (b). The feasible (green) and infeasible (red) samples/region during tuning based on the ordering constraint `min_wal_size` < `max_wal_size`.

operational cost but also slow model convergence. ML-based optimizers rely on observed performance signals to update surrogate models; invalid or low-quality configurations provide little useful information. As a result, substantially more iterations are required before the model forms reliable predictions, particularly in high-dimensional and constraint-heavy configuration spaces.

M2: Domain knowledge exists but remains underutilized. Extensive DBMS tuning knowledge has accumulated over decades and is documented in manuals, best practices, and expert guidelines. Prior work has incorporated portions of this knowledge into automated tuning frameworks, including suggested values [44, 45], range constraints, and special-value handling [24–26]. But, explicit *inter-knob dependency relationships* remain largely unmodeled. These dependencies directly affect both configuration validity and achievable performance. Existing ML-based tuners implicitly attempt to learn such relationships from sampled observations, resulting in high sample complexity and slow convergence in high-dimensional domains. There is no guarantee that a learned surrogate model faithfully captures structural knob relationships.

As illustrated in Figure 1, explicitly modeling knob dependencies significantly improves tuning efficiency. Incorporating constraints yields approximately 21% higher throughput within the first 100 iterations, demonstrating faster convergence. Furthermore, enforcing constraints such as `min_wal_size` ≤ `max_wal_size` eliminates nearly 50% of the nominal search region and reduces invalid samples by 19.5% during tuning. These results highlight that dependency constraints meaningfully shrink the effective search space and reduce wasted trials. These observations motivate a systematic mechanism for modeling and enforcing structural inter-knob dependencies during optimization.

M3: LLMs enable dependency extraction but introduce reliability challenges. DBMS dependency constraints are expressed in diverse linguistic forms and are often scattered across documentation, making direct extraction challenging. Relation type and direction are easily confused, especially when multiple knobs interact. Extraction errors are high-impact: hallucinated or incorrectly oriented constraints may prune feasible configurations or bias optimization. To address this, we design a two-part solution (Section 6): (i) a precision-first hybrid extraction core that combines schema-constrained LLM parsing with symbolic normalization and rule-based patterns, and (ii) an agentic reliability guardrail that applies self-reflection and LLM-as-judge verification before final conflict resolution. This design improves extraction trustworthiness while remaining lightweight and practical.

M4: Structural constraints require dedicated integration into optimization. Modern optimization frameworks such as BO and RL are designed for black-box search over box-constrained domains. While effective for many tuning tasks, this abstraction does not directly account for deterministic inter-knob dependencies that induce structured feasible regions. Thus, structural constraint information must be incorporated through explicit modeling choices rather than relying solely on standard optimizer interfaces. This motivates a constraint-aware optimization framework that integrates structural dependency modeling directly into the search process.

Together, these observations suggest that effective DBMS tuning requires enforcing documented knob dependencies directly within the optimization process. We formalize this constraint-aware tuning problem next.

3 PROBLEM FORMULATION

In this section, we formalize the constraint-aware DBMS knob configuration problem. We first revisit the classical formulation used in prior tuning work, and then introduce structural knob dependency constraints derived directly from DBMS manuals. We conclude by discussing the computational complexity of the resulting problem.

Classical DBMS Knob Tuning. Let $\theta = (\theta_1, \ldots, \theta_n)$ denote the configuration vector, where each knob θ_i takes values in domain Θ_i (continuous, integer, or categorical). The nominal configuration space is $\boldsymbol{\Theta} = \Theta_1 \times \cdots \times \Theta_n$.

Given a workload W, the DBMS performance (e.g., throughput or latency) is modeled as an unknown function $f : \boldsymbol{\Theta} \to \mathbb{R}$, which can only be evaluated by executing the workload under configuration θ. The classical tuning problem is therefore defined as [48]:

$$\theta^* = \arg\max_{\theta \in \boldsymbol{\Theta}} f(\theta). \tag{1}$$

Most existing tuning approaches treat $\boldsymbol{\Theta}$ as a box-bounded search space and perform black-box optimization over this space.

Constraint-aware Tuning with Ordering Dependencies. DBMS manuals document deterministic ordering relationships between knobs. These dependencies specify relative constraints such as $\theta_i \le \theta_j$, ensuring valid configurations and preventing unsupported parameter combinations. Such relationships are known *a priori* from documentation and can be verified without executing the DBMS (Table 1). They restrict the set of admissible configurations before any performance evaluation is performed. We model these dependencies as a set of pairwise ordering constraints:

$$\theta_i \le \theta_j, \quad (i, j) \in C, \tag{2}$$

where C denotes the set of ordered knob pairs extracted from documentation. The resulting feasible configuration space is

$$\boldsymbol{\Theta}_C = \left\{ \theta \in \boldsymbol{\Theta} \;\middle|\; \theta_i \le \theta_j \text{ for all } (i, j) \in C \right\}. \tag{3}$$

The constraint-aware tuning problem becomes:

$$\theta^* = \arg\max_{\theta \in \boldsymbol{\Theta}_C} f(\theta). \tag{4}$$

Unlike prior approaches that treat constraints as post-hoc performance conditions checked after execution (e.g., SLA violations), we enforce documented ordering dependencies directly in the search

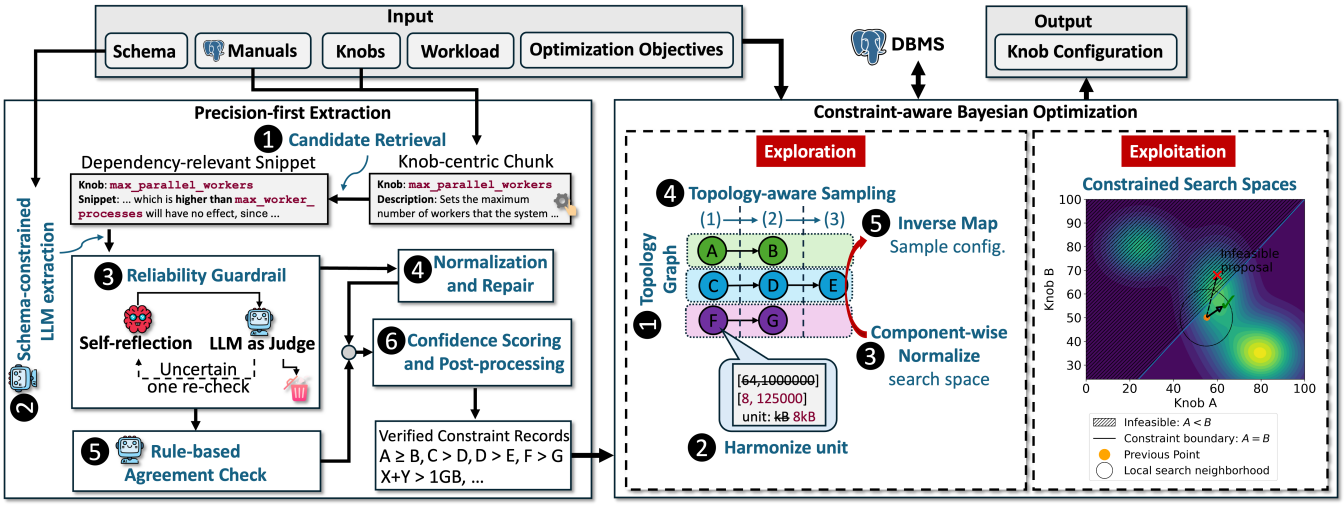

Figure 2: System overview of CATune. CATune consists of (1) a precision-first extraction pipeline that automatically derives ordering constraints from DBMS manuals, and (2) a constraint-aware Bayesian optimization engine that explicitly integrates these structural dependencies during both exploration and exploitation.

space. This eliminates structurally invalid configurations *a priori* and fundamentally changes how candidate configurations are generated during optimization.

3.1 Computational Complexity

Even when restricted to ordering constraints, the DBMS knob tuning problem remains computationally hard.

Lemma 3.1. *The constraint-aware DBMS knob tuning problem under a black-box objective is NP-hard.*

Proof Sketch. Consider a restricted instance in which all knobs are binary variables $\theta_i \in \{0, 1\}$ and no ordering constraints are imposed. Let the performance function be an arbitrary black-box mapping $f : \{0, 1\}^n \to \mathbb{R}$. In the worst case, identifying the global optimum requires evaluating all 2^n possible configurations. Thus, even without constraints, the tuning problem is combinatorial and NP-hard. Since adding ordering constraints only restricts the feasible region without introducing exploitable structure in f, the general problem remains NP-hard. □

Why the problem remains hard. The intrinsic difficulty stems from the objective function f, which is unknown, non-convex, and accessible only through costly workload executions. Ordering constraints reduce the feasible search region but do not impose structure on f itself. Consequently, global optimization still requires exploration of a combinatorial configuration space.

Feasibility checking. While optimization remains hard, checking whether the ordering constraints admit any feasible configuration is tractable. Pairwise ordering constraints of the form $\theta_i \leq \theta_j$ can be represented as a system of difference constraints. Detecting inconsistency reduces to cycle detection in the induced directed graph or, equivalently, to a shortest-path feasibility check (e.g., via Bellman–Ford) [4]. Thus, verifying that the constraint set is consistent can be done in polynomial time, even though optimizing over the feasible region is NP-hard.

4 CATUNE DESIGN

CATune is a constraint-aware DBMS tuning system that systematically incorporates documented structural knob dependencies into the optimization process. By enforcing these constraints during search, CATune shrinks the effective configuration space and reduces wasted trials.

Workflow. Figure 2 illustrates the overall workflow. The user provides (i) the target DBMS (e.g., PostgreSQL), (ii) the corresponding manuals, (iii) a set of tuning knobs with their search ranges (which can be obtained via existing techniques [25, 44, 55]), (iv) optimization objectives (e.g., throughput or latency), and (v) the target workload (e.g., TPC-H or TPC-C).

CATune first extracts knob dependency relationships from the manuals and refines them into machine-readable constraints (e.g., JSON format). It then performs constraint-aware BO to explore only feasible configurations and iteratively identify improved settings until the user-specified resource budget (e.g., maximum time or number of trials) is exhausted.

Components. CATune consists of two major components: (1) precision-first extraction (Section 6), and (2) constraint-aware Bayesian Optimization (Section 5).

Precision-first Extraction. To safely inject documentation-driven dependencies into tuning, CATune adopts a precision-first extraction pipeline that favors reliable abstention over aggressive recall. Motivated by the reliability risks discussed in M3, the module couples (i) schema-constrained LLM parsing with symbolic normalization and high-precision rule templates, and (ii) an agentic guardrail that applies self-reflection and LLM-as-judge verification before conflict resolution. The output is a set of machine-readable constraint records (e.g., knob pair, relation type, optional activation condition, and evidence span) that can be deterministically validated and directly consumed by the downstream optimizer.

Constraint-aware BO. CATune modifies standard BO to enforce structural dependencies during both exploration and exploitation.

Exploration phase. Sampling under cross-knob constraints leads to low acceptance rates and wasted trials. To generate feasible configurations, CATune performs five steps: ❶ constructs a topology graph encoding knob dependency relationships; ❷ harmonizes heterogeneous units across knobs to enable consistent comparisons; ❸ applies component-aware min–max normalization to transform knobs into a comparable search space; ❹ performs topology-aware sampling in the normalized space, ensuring all generated configurations satisfy active constraints without rejection [2]; ❺ inverse-maps normalized samples back to their original value domains.

Exploitation phase. During exploitation, ordering and dependency constraints explicitly restrict the feasible region considered by the acquisition function. The surrogate model proposes new trials only within this constrained space, preventing evaluations in infeasible regions and improving sample efficiency.

5 CONSTRAINT-AWARE BAYESIAN OPTIMIZATION

In this section, we describe how CATune modifies standard BO to enforce structural knob dependencies during both exploration and exploitation. The key idea is to restrict the optimizer to valid configurations *by construction*, thereby eliminating infeasible evaluations and improving sample efficiency. Algorithm 1 outlines the overall procedure. We redefine the feasible configuration domain using the extracted dependency constraints (Line 4), so that both exploratory sampling (Line 14) and acquisition maximization (Line 16) operate only over valid configurations. Hence, the optimizer no longer wastes trials on structurally invalid settings.

5.1 Exploitation Phase

During exploitation, we restrict acquisition maximization to the feasible configuration space defined by the dependency constraints. Rather than modifying the surrogate model itself, we enforce constraint validation during candidate selection. Specifically, when maximizing the acquisition function within a local neighborhood $\mathcal{N}(\boldsymbol{\theta}_t)$, candidate configurations that violate any dependency constraint are discarded. Thus, acquisition maximization is effectively performed over

$$\mathcal{N}(\boldsymbol{\theta}_t) \cap \Theta_C,$$

where Θ_C denotes feasible space restricted by dependency constraints.

Figure 2 (Exploitation) illustrates this process under the ordering constraint $A \geq B$. While the unconstrained acquisition direction may point toward an infeasible region ($A < B$), constraint enforcement restricts the step to the admissible portion of the neighborhood. This ensures that all exploitation steps remain valid without altering the surrogate model.

5.2 Exploration Phase

To enforce ordering constraints during the exploration phase, we design a five-step procedure that systematically ensures constraint consistency throughout the search process. Figure 3 illustrates an end-to-end example of this workflow.

5.2.1 Topology Graph. Ordering constraints naturally induce a dependency structure among knobs. We represent this structure as a directed graph $G = (V, E)$, where each knob is a node and each ordering constraint defines a directed edge. For a constraint (a, op, b), the edge direction reflects the sampling dependency implied by the operator. If $\text{op} \in \{<, \leq\}$ (i.e., a must not exceed b), we add an edge $b \to a$, indicating that b must be assigned before a. If $\text{op} \in \{>, \geq\}$, we add an edge $a \to b$, meaning that a must be assigned prior to b. Algorithm 2 describes this construction.

The resulting graph explicitly captures sampling dependencies among knobs and enables constraint-aware generation of configurations via topological traversal (Section 5.2.3). After inserting all edges, we check for directed cycles. A cycle indicates mutually inconsistent ordering constraints and thus an invalid constraint set. Otherwise, the graph forms a directed acyclic graph (DAG) that

Algorithm 1: Constraint-Aware Bayesian Optimization for DBMS Tuning (differences to standard BO in blue).

Input: Database system DB; nominal configuration space Θ; constraints C; performance metric $f(\cdot)$; objective type obj $\in$ {throughput, latency}; initial design size n_0; evaluation budget T.

Output: Best configuration $\boldsymbol{\theta}^*$ `// Best among trials`

1 Let $\mathcal{D} \leftarrow \emptyset$; `// Tuning trajectories`
2 Let $G \leftarrow$ `BuildTopoGraph`$(C, \boldsymbol{\theta})$; `// Build topology graph, see Algorithm 2`
3 $\widehat{\Theta} =$ `Normalize`(Θ, G); `// See Alg. 3`
4 Construct feasible space $\widehat{\Theta}_C \triangleq \{\hat{\theta} \in \widehat{\Theta} \mid c(\hat{\theta}) = \text{True},\ \forall c \in C\}$;
5 **(1) Feasible initial design.**;
6 $\{\hat{\boldsymbol{\theta}}_1, \ldots, \hat{\boldsymbol{\theta}}_{n_0}\} \leftarrow$ `TopoSampler`$(\widehat{\Theta}, G, n_0)$; `// Alg. 4`
7 **for** $i = 1$ **to** n_0 **do**
8 Apply $\hat{\boldsymbol{\theta}}_i$ to DB and observe $y_i \leftarrow f(\hat{\boldsymbol{\theta}}_i)$;
9 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\hat{\boldsymbol{\theta}}_i, y_i)\}$;
10 **(2) Iterative BO over the feasible space.**;
11 **for** $t = n_0 + 1$ **to** T **do**
12 $\mathcal{M} \leftarrow$ `Train`$(\mathcal{D})$; `// train surrogate on feasible data only`
13 **if** rand$(0, 1) < p$ **then**
14 $\hat{\boldsymbol{\theta}}_t \leftarrow$ `TopoSampler`$(\widehat{\Theta}, G, 1)$; `// feasible exploration`
15 **else**
16 $\hat{\boldsymbol{\theta}}_t \leftarrow$ `SuggestNext`$(\mathcal{M}, \widehat{\Theta}_C, \mathcal{D})$; `// maximize acquisition within` $\widehat{\Theta}_C$
17 $\boldsymbol{\theta}_t \leftarrow$ `InverseMap`$(\hat{\boldsymbol{\theta}}_t)$; `// See Eq. 8 and 9`
18 Apply $\boldsymbol{\theta}_t$ to DB and observe $y_t \leftarrow f(\boldsymbol{\theta}_t)$;
19 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\boldsymbol{\theta}_t, y_t)\}$;
20 **if** obj = throughput **then**
21 $\boldsymbol{\theta}^* \leftarrow \arg\max_{(\boldsymbol{\theta}, y) \in \mathcal{D}} y$;
22 **else**
23 $\boldsymbol{\theta}^* \leftarrow \arg\min_{(\boldsymbol{\theta}, y) \in \mathcal{D}} y$;
24 **return** $\boldsymbol{\theta}^*$;

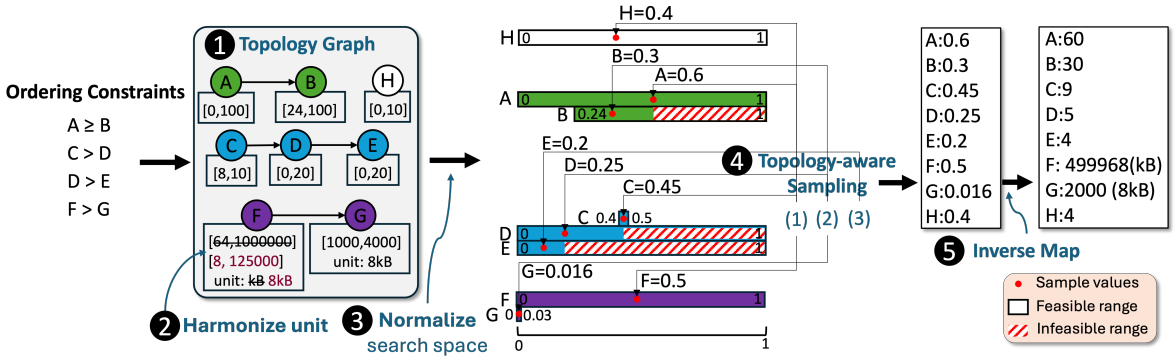

Figure 3: End-to-end illustration of the exploration phase. Given four ordering constraints, including a chained dependency ($C > D > E$), we construct the topology graph, where independent knob H appears as an isolated node. Heterogeneous units (e.g., F and G) are harmonized to enable consistent comparison, and each connected component (color-coded) is normalized via component-wise min–max normalization while preserving relative structure. Topology-aware sampling generates a feasible configuration in three steps by respecting dependency order and feasible ranges. Finally, normalized values are mapped back to knob units through inverse transformation.

defines a partial order over dependent knobs, while independent knobs remain isolated nodes, as illustrated in Figure 3. This representation separates constraint reasoning from value sampling, allowing the optimizer to enforce dependencies without modifying the surrogate model.

Algorithm 2: Build a Topology Graph from Ordering Constraints

Input: ordering constraint list $\mathcal{R} = \{(a, \text{op}, b)\}$, where $\text{op} \in \{<, \leq, >, \geq\}$; knob set $\mathcal{K}$.
Output: Directed graph $G = (V, E)$ with edge labels storing op.

$V \leftarrow \emptyset, E \leftarrow \emptyset$;
foreach $(a, \text{op}, b) \in \mathcal{R}$ **do**
  $V \leftarrow V \cup \{a, b\}$;
  if $\text{op} \in \{<, \leq\}$ **then**
    $E \leftarrow E \cup \{(b \rightarrow a, \text{op})\}$;
  else
    $E \leftarrow E \cup \{(a \rightarrow b, \text{op})\}$;
// Append independent knobs
foreach $k \in \mathcal{K}$ **do**
  if $k \notin V$ **then**
    $V \leftarrow V \cup \{k\}$;
if G *contains a directed cycle* **then**
  return FAIL;
return G;

5.2.2 Search Space Normalization. Ordering constraints require value comparison across knobs. However, DBMS knobs often use heterogeneous units. For example, `shared_buffers` may be specified in pages, while `max_wal_size` is specified in MB, making raw comparisons invalid. To ensure consistent constraint enforcement, we perform two preprocessing steps: (i) unit harmonization and (ii) component-wise normalization (Algorithm 3).

Algorithm 3: Search Space Normalization

Input: Knobs $\mathcal{K} = \{\theta_i\}$ with domains $[\ell_i, u_i]$ and unit unit_i; topology graph G
Output: Normalized domains $\{\widehat{\Theta}_i\}$

foreach $\mathcal{G}_k$ *in* G **do**
  foreach $\theta_i \in \mathcal{G}_k$ **do**
    1. Unit Harmonization;
    $\tilde{\theta}_i \leftarrow Harmonize(\theta_i)$ `// Eq. 5`
    2. Component-wise min-max Normalization;
    $\ell^{(k)}, u^{(k)} = \texttt{SharedInterval}(\tilde{\theta}_i, \mathcal{G}_k)$ `// Equation 6`
    $\widehat{\Theta}_i \leftarrow \texttt{MinMaxNormalize}(\tilde{\theta}_i)$`// Eq. 7`
return $\{\widehat{\Theta}_i\}$;

Unit Harmonization. For each semantic knob type (e.g., memory, time), CATune convert all knobs to a canonical unit. For a knob θ_i with domain $\Theta_i = [\ell_i, u_i]$ and unit unit_i, let $\text{unit}^{(t)}$ denote the canonical unit for its type t. We apply a deterministic linear conversion:

$$\tilde{\theta}_i = \phi_i(\theta_i) = \alpha_i \theta_i, \qquad \tilde{\Theta}_i = [\alpha_i \ell_i, \alpha_i u_i], \tag{5}$$

where α_i is the deterministic conversion factor from unit_i to $\text{unit}^{(t)}$.

Component-wise Normalization. Since ordering constraints connect knobs that must be numerically comparable (after unit harmonization), each connected component defines a group of knobs sharing a common comparison space. For a component $\mathcal{G}_k$, we define the shared interval:

$$\ell^{(k)} = \min_{\theta_i \in \mathcal{G}_k} \tilde{\ell}_i, \qquad u^{(k)} = \max_{\theta_i \in \mathcal{G}_k} \tilde{u}_i, \tag{6}$$

and normalize each knob $\theta_i \in \mathcal{G}_k$ as

$$\hat{\theta}_i = \frac{\tilde{\theta}_i - \ell^{(k)}}{u^{(k)} - \ell^{(k)}}. \tag{7}$$

Different components are normalized independently.

Example 1. *Suppose* `work_mem` $\leq$ `shared_buffers` *with domains* $[4, 512]$ *MB and* $[128, 8192]$ *MB respectively. Independent normalization would map both domains to* $[0, 1]$*, distorting their relative scale. Component-wise normalization instead uses the shared interval* $[4, 8192]$*, preserving ordering consistency in normalized space.*

Inverse Mapping. Before applying a configuration to the DBMS, we invert the preprocessing steps. For $\hat{\theta}_i \in [0, 1]$ in component $\mathcal{G}_k$,

$$\tilde{\theta}_i = \hat{\theta}_i \left(u^{(k)} - \ell^{(k)} \right) + \ell^{(k)}, \tag{8}$$

and the original knob value is recovered as

$$\theta_i = \tilde{\theta}_i / \alpha_i. \tag{9}$$

Example 2. *Suppose* `work_mem` *belongs to a connected component with shared interval* $\ell^{(k)} = 4$ *MB and* $u^{(k)} = 8192$ *MB. Assume its harmonization factor is* $\alpha_i = 1$ *(already expressed in MB). If the optimizer proposes a normalized value* $\hat{\theta}_i = 0.5$*, the harmonized value is recovered via Equation* (8)*:*

$$\tilde{\theta}_i = 0.5 \times (8192 - 4) + 4 = 4098 \text{ MB}.$$

Since $\alpha_i = 1$*, the original knob value is*

$$\theta_i = 4098.$$

If instead the knob were originally specified in pages (e.g., 1 page = 8 KB) and α_i *converted pages to MB, Equation* (9) *would restore the value to its native unit before applying it to the DBMS. This design allows* CATune *to operate in a unit-consistent normalized space during optimization, while ensuring that deployed configurations remain expressed in their native DBMS units. By decoupling optimization from unit heterogeneity, we preserve semantic correctness without complicating constraint enforcement.*

5.2.3 Topology-aware Sampling. To enable efficient exploration of the feasible region, we propose a topology-aware sampler that eliminates the inefficiencies of rejection-based constrained sampling.

Motivation. Compared to straightforward ways to enforce ordering constraints (details in Section 9.3), for example, a rejection-based strategy: repeatedly draw a configuration from the full Cartesian product and discard it if any constraint is violated [17]. While simple, this approach becomes impractical when the number of ordering constraints increases, especially when they are chained, and their search spaces are extremely imbalanced. In such cases, the feasible region may occupy only a small fraction of the full search space, and the probability of sampling a valid configuration drops multiplicatively as constraints accumulate. As a result, the optimizer may require an excessive number of trials to obtain a single feasible configuration.

Case Study. Consider the constraints `superuser_reserved_connections` (range: [0, 262143]) < `max_connections` (range: [24, 100]) and `max_wal_senders` (range: [2, 8000]) < `max_connections`. Due to highly imbalanced value ranges, the probability of randomly satisfying each constraint can be extremely small. When combined, the joint acceptance probability may fall to around 10^{-6}, implying millions of random trials per feasible sample. Such inefficiency severely degrades the optimization process.

Topology-Aware Sampling. To avoid rejection, we generate configurations directly in a dependency-consistent order defined by the topology graph [7, 31]. We first identify root knobs (nodes with no incoming edges) and sample them using Latin Hypercube Sampling (LHS) to ensure good coverage of their domains [19, 22, 42, 56]. We then traverse the graph in topological order. When sampling a knob, we dynamically restrict its feasible interval based on the already assigned values of its parent knobs. Thus, every sampled value automatically satisfies all ordering constraints.

This sequential, constraint-aware sampling eliminates rejection entirely: every generated configuration is feasible by construction. Moreover, sampling cost grows linearly with the number of knobs rather than being governed by the shrinking acceptance probability of the full Cartesian space.

Figure 3 illustrates this process. Root nodes are sampled first. Subsequent knobs are sampled within dynamically restricted intervals determined by their parents, progressively pruning infeasible regions. After all knobs are assigned, inverse normalization restores values to their original DBMS units.

Algorithm 4: Topological Sampler

Input: Configuration space C with d knobs; topology graph $G = (V, E)$; number of samples N.
Output: A set of valid configurations $\mathcal{S} = \{\mathbf{c}_1, \ldots, \mathbf{c}_N\}$.

Compute a topological order $\pi \leftarrow \text{TopoSort}(G)$
Let $\mathcal{R} \leftarrow \{v \in V \mid \text{indegree}(v) = 0\}$ `// root knobs and independent knobs`
Initialize $\mathcal{S} \leftarrow \emptyset$ `// sample set`
`// (1) Sample roots`
Sample $\mathbf{U} \in [0, 1]^{N \times |\mathcal{R}|}$ for $\mathcal{R}$ using LHS;
for $i = 1 \ldots N$ **do**
    Initiate an empty configuration $\mathbf{c}_i$;
    for $j = 1 \ldots |\mathcal{R}|$ **do**
        $\mathbf{c}_i[hp_j] \leftarrow \text{convert}(u_{ij}, hp_j)$
`// (2) Sample remaining knobs in topological order`
for $i = 1$ **to** N **do**
    foreach $x \in \pi \setminus \mathcal{R}$ **do**
        Initialize feasible space $I_x = \text{def\_space}(x) = [l_x, h_x]$
        foreach *incoming edge* $(p \to x, \text{op}) \in E$ **do**
            $h_x \leftarrow \min(h_x, \mathbf{c}_i[p])$ `// Tighten` I_x
        Sample $\mathbf{c}_i[x]$ uniformly from I_x
    $S \leftarrow S \cup \{\mathbf{c}_i\}$
return $\mathcal{S}$

6 KNOB CONSTRAINT EXTRACTION

CATune requires reliable knob dependencies before they can be incorporated into the configuration search space. However, these dependencies are expressed in heterogeneous forms across DBMS documentation, and an incorrectly extracted relation may exclude valid configurations. We therefore develop a precision-first pipeline that extracts structured dependencies from manuals while filtering unsupported and ambiguous relations. As shown in Section 7.2, direct LLM prompting can produce unsupported or incorrectly

oriented dependencies even when evidence spans are requested, motivating the reliability safeguards introduced below.

6.1 Extraction Scope and Inputs

The extraction module takes three inputs: (1) a version-specific corpus of official DBMS documentation, (2) a target knob list, and (3) a predefined relation schema. The knob list can be obtained using existing knob selection techniques, such as SHAP [**?**], Lasso [**?**], advanced LLM-assisted knob selection approaches [25], or manual selection. Knob selection itself is outside the scope of this paper. The relation schema is specified according to the dependency types represented by the extraction layer. This preparation is performed once for each DBMS. Given these inputs, candidate retrieval, extraction, verification, normalization, and filtering are performed automatically. Section 7.2 evaluates the resulting records under a common extraction protocol. Each output record contains `(knob1, relation, knob2, condition, context, evidence, confidence)`.

The current topology-aware optimizer consumes verified, unconditional pairwise numerical ordering constraints, such as $A < B$ and $A \leq B$. The extraction schema can additionally retain activation conditions, resource bounds, multi-knob dependencies, and performance recommendations, but the current sampler does not automatically convert these richer relations into topology edges. Section 7.1 characterizes these relations and makes the distinction between extraction coverage and optimizer support explicit.

Our design prioritizes precision because extraction errors have asymmetric downstream effects. A missed constraint leaves part of the original search space unpruned, whereas a hallucinated or incorrectly oriented constraint may remove valid and potentially high-performing configurations, as demonstrated in Section 9.1. Consequently, CATune retains only evidence-supported relations and abstains when the documentation does not provide sufficient support. Its scope is intentionally limited to dependencies stated in the selected manual version: it does not discover undocumented, workload-specific, or cross-version dependencies unless they are supplied through an additional evidence source.

6.2 Precision-First Extraction Pipeline

Figure 2 presents the extraction pipeline. It combines schema-constrained LLM extraction with evidence verification, canonical normalization, and deterministic rules for recurring documentation expressions.

(1) Candidate retrieval. We parse the documentation into knob-centric entries and identify dependency-relevant paragraphs using knob co-mentions and trigger expressions, such as "must be less than," "at least," "ignored unless," and "has no effect" [32, 37, 39]. The trigger expressions are derived from the closed relation schema and fixed before extraction. A trigger is used only for retrieval and never establishes a constraint by itself. Each selected paragraph is expanded with its local context and divided into bounded-size chunks. This step reduces irrelevant input while preserving the context needed to interpret the relation.

(2) Schema-constrained extraction. For each candidate chunk, the LLM receives the primary knob, the knobs mentioned in the local context, and a closed set of relation labels. It must return structured JSON records and provide a supporting evidence span for every extracted relation. If the chunk contains no explicit dependency, the model is instructed to return an empty result. Requiring source-grounded evidence improves traceability and supports deterministic verification [16, 29].

(3) Evidence verification. Each preliminary record passes through two verification stages. Self-Reflection may revise or remove unsupported records and correct relation, direction, or condition errors [18, 41]. A separate LLM-as-Judge call, using a verification-specific prompt but the same model backend by default, then scores the revised record on three *label-agnostic* axes: whether the snippet states the dependency at all, whether the two operands are in the correct roles for the relation, and whether any activation condition is faithful [51, 60]. A record is rejected if any axis fails. The judge deliberately does not arbitrate which relation label is "most accurate": the documentation's labelling is not unique (an inequality may also be described associatively), so penalizing label choice steers extraction away from the reference rather than toward it; verifying only support, direction, and condition is what yields a precision gain without fitting the reference. Records with uncertain support trigger one additional judge call and are retained only when the two judge calls agree on the same canonical relation; otherwise, the pipeline abstains.

(4) Normalization and semantic repair. We normalize knob names, relation labels, activation conditions, and relation directions. Semantically equivalent descriptions are mapped to the same representation. For example, both "A must not exceed B" and "B must be at least A" are normalized to $A \leq B$. This canonicalization prevents equivalent statements from producing oppositely oriented graph edges.

(5) Deterministic rules. Generic comparison templates handle recurring documentation expressions, such as mapping "must be less than X" to a strict ordering relation. The rule branch provides an additional evidence signal for explicit patterns, while the extracted record still passes through evidence verification and canonical normalization.

(6) Scoring and post-processing. A record is retained when it passes the judge's support, direction, and condition checks (Stage 3). Reflection consistency, judge support, and rule agreement are combined into a reliability score used to rank duplicate or conflicting records. We then deduplicate identical tuples and resolve conflicting relations for the same directed knob pair by retaining the best-supported record. The resulting constraints are passed to topology construction, where inconsistent directed cycles are detected as described in Algorithm 2.

Example 3. *Consider the documentation statement that a value of* `max_parallel_workers` *higher than* `max_worker_processes` *has no effect [37]. For readability, let A and B denote these two knobs, respectively. Candidate retrieval selects this statement based on the knob co-mention and the phrase "higher than." The LLM branch extracts the relation and its evidence span, which normalization canonicalizes to $A \leq B$. The verification stage then confirms that the snippet states the dependency, that A and B occupy the correct roles, and that no spurious condition was added, so the record is retained. A record that failed any of these checks—for example, one asserting the reverse ordering—would be rejected.*

6.3 Difference from Existing Manual-Reading Systems

Our extraction objective differs from those of existing manual-reading tuning systems. Prior systems extract textual hints that recommend values for individual knobs and adapt or aggregate these recommendations using runtime feedback, or build per-knob structured knowledge such as suggested, minimum, maximum, and special values [24, 44]. Such knowledge is primarily used for knob selection, range refinement, and tuning initialization.

In contrast, CATune extracts *inter-knob dependencies* rather than recommended values for individual knobs. Each output explicitly records the participating knob pair, relation direction, optional activation condition, and supporting evidence. These records can therefore be converted into structural constraints on the configuration space when their relation type is supported by the downstream optimizer.

The reliability mechanisms also reflect this different objective. Because an incorrect dependency may remove valid configurations, CATune restricts extraction to the provided documentation, requires evidence spans, verifies each relation through separate reflection and judge calls, and canonicalizes relation directions before graph construction. Therefore, our extraction pipeline complements rather than replaces value-oriented manual-reading knowledge. Section 7.2 evaluates extraction strategies designed for inter-knob constraints under a common protocol.

7 CONSTRAINT CHARACTERIZATION AND EXTRACTION QUALITY

This section characterizes the dependency relations found in DBMS documentation, clarifies which relation types are represented and enforced by CATune, and evaluates the quality of automatic extraction. We deliberately separate three questions: what relations occur in manuals, which relations the current optimizer can enforce, and how accurately different manual-reading strategies recover them.

7.1 Constraint Types and Supported Scope

DBMS manuals contain more than pairwise numerical inequalities. Besides pairwise ordering constraints, they describe activation or assignment dependencies, coordinated recommendations, multi-knob constraints, and bounds tied to system resources or platform characteristics [9, 32, 37, 39]. Table 2 summarizes the major constraint categories observed in DBMS documentation and their current treatment in CATune.

Structural coverage. We audit dependency-bearing statements from the official configuration documentation of PostgreSQL, MySQL, and ClickHouse [8, 9, 33, 38]. We retain only statements describing runtime dependencies between settings or between a setting and an external system property. Each statement is assigned to one structural category according to its primary semantics. Duplicate descriptions are merged, and multi-knob constraints are counted once.

For each system, we report the number of retained relations in each category and the fraction represented as pairwise ordering constraints:

$$\text{Share}_{\text{pair}} = \frac{N\text{ordering}}{N_{\text{all}}} \tag{10}$$

This ratio measures the structural coverage of the topology representation.

Table 3 reports the resulting audit catalog. The counts are obtained from manually verified documentation evidence rather than extraction outputs. For MySQL, we additionally inspect the variable-definition pages linked from the MySQL8.0 System Variable Index and merge deprecated aliases with their replacement names [33]. The catalog characterizes audited dependency evidence and is not intended as an exhaustive census of each manual.

Binding requirements versus advisory suggestions. Structural form is distinct from operational impact. A *binding requirement* is stated as a mandatory condition or describes a configuration that is invalid, ineffective, or disables required functionality. An *advisory suggestion* recommends coordinated values for performance but does not make other configurations invalid [37, 39]. Extraction confidence measures whether the documentation supports a relation; it does not determine whether that relation is binding. We therefore verify operational semantics against the cited evidence before a relation is enforced. The current optimizer supports verified, binding, unconditional pairwise orderings and resource-bounded constraints that can be expressed as upper-bound limits. Advisory, conditional, and multi-knob relations are retained rather than silently treated as hard inequalities. The broader catalog provides extension points beyond the implemented handler. Conditional relations require gated edges or pre-execution validators; Advisory suggestions could instead guide proposal ranking or a soft penalty without excluding configurations. Table 3 therefore reports these structural classes separately rather than implying that every extracted record is currently enforced.

7.2 Extraction Quality and Baseline Comparison

Reference catalog. We evaluate extraction on the PostgreSQL 13 configuration manual using a fixed list of 49 target knobs and a manually verified reference catalog. Each reference record contains two knobs or operands, a canonical relation, an optional activation condition, and its supporting manual text.

Experimental setup. Candidate retrieval produced 53 chunks using two neighboring paragraphs and a maximum chunk length of 1,800 characters. All methods use `deepseek-4-pro` at temperature 0 with the same corpus, knob scope, candidate chunks, and relation schema. The direct baseline makes one schema-constrained call per chunk but performs no reflection or judging. We define a *Prompt-Ensemble* baseline to isolate the effect of repeated prompting and voting: it uses a fixed pool of ten synthetic demonstrations with placeholder knob names, samples three demonstrations per prompt, issues five schema-constrained prompts per candidate chunk, normalizes each output to the common tuple schema, and retains a tuple only if it receives a strict majority of at least three votes. After chunk-level voting, it applies only common tuple deduplication and best-record selection for duplicate directed pairs; it does not use self-reflection, LLM-as-judge verification, deterministic rule extraction, or the precision filters used by CATune. CATune uses the complete pipeline in Section 6.2, including evidence verification and canonical normalization.

Table 2: Constraint types represented by the extraction layer and their current use in CATune. "Represented" means that the evidence and relation can be retained as a structured record; it does not imply that every extracted relation is converted into a topology constraint.

Structural form	Documentation example	Extracted representation	Current optimizer support
Pairwise ordering (Order.)	A must not exceed B	Directed relation $(A, \leq, B)$	Supported and enforced during topology-aware sampling.
Activation/assignment (Act.)	A is ignored unless B is enabled; The maximum effective value of A is lesser of the value of B and c	Dependency relation with activation or assignment semantics	Extracted but not currently enforced.
Coordinated recommendation (Coor.)	Increasing A usually requires increasing B	Advisory relation such as `requires-larger`	Retained as guidance; not enforced.
Functional/multi-knob (Func.)	$A \times B < C$; configure A, B, and C jointly	Constraint function or operand graph	Partially represented; not currently enforced.
Resource/platform-bounded (Res.)	Maximum value of A depends on available memory or OS architecture	Relation between a knob and an external system property	Retained separately from the knob topology; expressed as upper-bound limits.

Table 3: Manually verified statement-level audit catalog of documented dependency constraints. $\text{Share}_{\text{pair}}$ is the fraction of audited dependencies represented as pairwise ordering constraints. The catalog characterizes the audited evidence and is not intended as an exhaustive census of each manual.

DBMS	Order.	Act.	Coor.	Func.	Res.	Total	$\text{Share}_{\text{pair}}$
PostgreSQL	11	13	6	5	0	35	31.4%
MySQL	10	11	2	4	0	27	37.1%
ClickHouse	1	25	1	18	3	48	2.1%

Table 4: Held-out test-split constraint-extraction accuracy. All methods use the same documentation corpus, candidate chunks, relation schema, and LLM backend.

Method	Precision	Recall	F1
Naive LLM	0.792	**0.864**	0.826
Prompt-Ensemble	0.773	0.773	0.773
CATune	**0.950**	**0.864**	**0.905**

Comparison methods. We quantitatively compare three methods that emit the same inter-knob tuple schema: *Naive LLM*, *Prompt-Ensemble*, and the complete CATune pipeline. Native value-oriented manual-reading outputs do not directly encode a directed inter-knob tuple [24, 44]; assigning precision and recall to those incompatible outputs would therefore be misleading. The Prompt-Ensemble baseline is therefore a controlled baseline defined for this study, not an implementation of a prior tuning system.

Metrics. We partition the reference catalog into development and held-out test splits by a deterministic hash of the canonical knob pair, keeping all relations for the same pair in the same split. Prompt and verification choices are fixed using the development split only, and Table 4 reports precision, recall, and F_1 on the held-out test split. A predicted tuple is matched using its knob pair, canonical relation, and normalized direction; operand order is canonicalized for explicitly unordered association types.

Extraction quality. Table 4 shows that direct prompting and Prompt-Ensemble achieve held-out F_1 scores of 0.826 and 0.773, respectively. In comparison, the complete CATune pipeline achieves precision 0.950, recall 0.864, and held-out F_1 0.905. Relative to Prompt-Ensemble, CATune improves F_1 by 13.2 absolute percentage points. This result shows that prompt ensembling alone is insufficient for reliable inter-knob constraint extraction, whereas the evidence verification and normalization stages in CATune substantially reduce extraction errors.

7.3 Implications for Constraint-Aware Tuning

Extraction errors affect tuning in different ways. A false negative leaves a documented dependency unenforced and therefore approaches the unconstrained baseline for that portion of the search space. A false positive or direction error is more harmful: it introduces an unsupported boundary and may remove valid, high-performing configurations. To separate extraction quality from optimizer behavior, Table 4 evaluates extraction independently, while Section 9.1 provides the controlled downstream study.

That study compares tuning without injected constraints, tuning with manually verified constraints, and tuning after adding two hallucinated ordering constraints. With verified constraints, CATune reaches the unconstrained baseline optimum 4.55× faster and improves final performance by 59.15%. Adding the two incorrect constraints reduces the achievable improvement by 32.07%; relative to the hallucinated setting, the verified setting obtains 134.29% higher final performance and converges 9.09× faster (Figure 10). These results quantify both ends of the extraction trade-off: missing constraints forgo available pruning, whereas unsupported constraints can incorrectly truncate the feasible region. This downstream study injects *manually verified* constraints (and two deliberately hallucinated ones) rather than the raw extractor output, so it is independent of the extractor's measured precision; its numbers

predate the extraction audit reported here and are not re-derived in this revision.[1]

The current study does not claim that every documented relation should be enforced. Only relations that are evidence-supported, operationally binding, and compatible with the direct unconditional pairwise representation are converted into topology edges. The broader catalog in Table 3 identifies where conditional, resource-aware, or multi-knob handlers would be required.

8 EVALUATION

This section evaluates the impact of ordering constraints on tuning efficiency and performance. We examine constraint-aware search across workloads, search-space regimes, and BO frameworks, and compare topology-aware sampling with rejection-based strategies.

8.1 Experimental Setup

Workloads. We evaluate our approach on two standard benchmarks covering analytical and transactional regimes: TPC-H (OLAP workload) with scale factor 1, and TPC-C (OLTP workload) with scale factor 200. TPC-C is configured with ten terminals and unlimited arrival rate to stress concurrency behavior. All benchmark implementations are from BenchBase [11]. These workloads are widely adopted in DBMS tuning studies [24].

Hardware. All experiments are conducted on a dedicated machine equipped with a 12-core 12th Gen Intel(R) Core(TM) i7-12700K CPU, 32 GB RAM, and a 1 TB Samsung SSD.

Tuning Settings. We conduct experiments using PostgreSQL v13 and MySQL v8.0. We assume a candidate set of tunable knobs is provided, since prior ML-based tuning systems treat knob selection as an independent preprocessing step (e.g., GPTuner and OpAdviser) [24, 58]. Our work focuses on discovering and enforcing structural dependencies within a given knob set, rather than on knob selection itself.

In our experiments, we consider 45 and 50 performance-relevant knobs for PostgreSQL and MySQL, respectively, spanning memory allocation, parallelism, and write-ahead logging behavior. We exclude parameters related to debugging, logging verbosity, security policies, and file path configuration, as these do not directly influence performance. From this knob set, we extract 10 documented ordering dependencies and enforce them in the constraint-aware setting.

Each workload is tuned for 200 iterations. The first 10 configurations are generated via LHS (baseline) or topology-aware sampling (our method) as the initial design. We perform four independent runs with different random seeds to account for optimizer stochasticity and report the mean best performance across runs. We optimize for overall system throughput (higher is better) for TPC-C and average latency (lower is better) for TPC-H, following common practice in prior tuning work [24].

Optimizers. We evaluate our method using two widely adopted Bayesian optimization frameworks: SMAC and Gaussian Process BO (GP-BO). Both are implemented within the SMAC3 infrastructure [30], ensuring a unified experimental environment. For SMAC, we use its default Random Forest surrogate model with expected improvement. For GP-BO, we replace the surrogate with a Gaussian Process while retaining the same acquisition optimization pipeline. This unified setup isolates the effect of surrogate modeling from implementation-level artifacts.

To assess compatibility with knowledge-enhanced tuning, we additionally evaluate integration with GPTuner, a coarse-to-fine BO framework built on SMAC [24, 30].

[1]The downstream tuning numbers were not re-run for this revision and should be re-verified separately before publication.

8.2 Efficiency Evaluation

Evaluation Dimensions. We evaluate CATune along two dimensions: (i) **final tuning performance** (best throughput or latency after 200 iterations), and (ii) **sample efficiency**, measured as the number of iterations required to reach the best observed performance (time-to-optimal). Unless otherwise stated, SMAC with a random forest surrogate serves as the primary baseline.

Scenarios. To isolate the effect of constraint-aware sampling from other forms of tuning knowledge, we consider four settings:

- *SMAC (default):* SMAC operating over the default knob ranges specified in the PostgreSQL manuals.
- *SMAC+CATune (default):* SMAC augmented with constraint-aware tuning under the same default ranges.
- *SMAC (suggest):* SMAC using knowledge-guided suggested ranges derived from GPTuner's knowledge handler [24].
- *SMAC+CATune (suggest):* Suggested ranges combined with constraint-aware tuning.

This allows us to evaluate (i) the standalone impact of ordering constraints and (ii) their interaction with range-based knowledge.

Results: Default Search Ranges. Figure 4 reports results under vendor-provided default ranges on PostgreSQL. CATune significantly improves sample efficiency, reaching the baseline SMAC optimum (after 200 iterations) 12.5× faster on TPC-C and 2.27× faster on TPC-H. It also yields superior final configurations: after 200 iterations, throughput improves by 33.39% on TPC-C and latency decreases by 12.55% on TPC-H relative to baseline SMAC. Figure 6 shows the corresponding results on MySQL. Although the final performance of CATune is comparable to that of SMAC on TPC-C, it reaches the baseline SMAC optimum 1.12× faster.

Results: Suggested Search Ranges. Figure 5 show results under knowledge-guided reduced ranges on PostgreSQL and MySQL. The gains remain consistent. CATune reaches the baseline SMAC optimum 5.41× faster on TPC-C and 3.57× faster on TPC-H. In terms of final performance, it improves throughput by 63.37% on TPC-C and reduces latency by 10.73% on TPC-H compared to baseline SMAC. On MySQL, CATune achieves an 11.35% higher throughput than baseline SMAC on TPC-C while reaching the baseline optimum 28.5× faster.

Discussion. Comparing default and knowledge-guided ranges disentangles the effect of ordering constraints from simple range narrowing. Default ranges represent a cold-start regime with large infeasible or low-quality regions, whereas reduced ranges correspond to a warm-start regime where prior knowledge already prunes part of the domain. The consistent improvements in both settings indicate that constraint-aware tuning provides benefits beyond range restriction alone. Overall, CATune improves both convergence speed and final tuning quality across deployment scenarios.

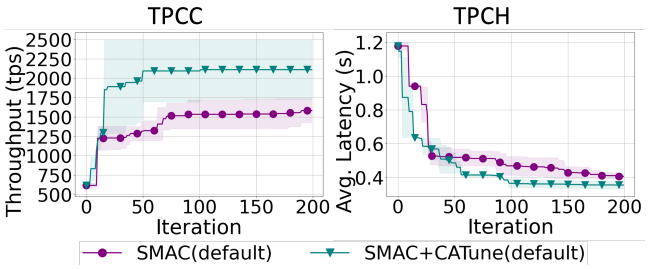

Figure 4: Best performance achieved by CATune under manual default ranges on PostgreSQL.

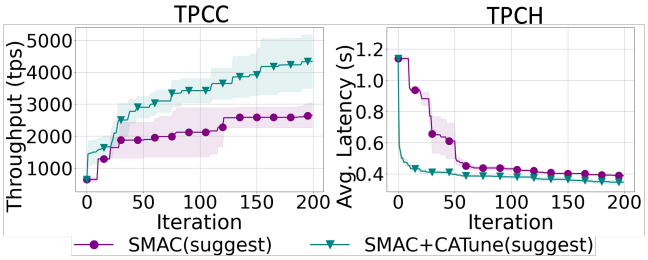

Figure 5: Best performance achieved by CATune under knowledge-guided reduced ranges on PostgreSQL.

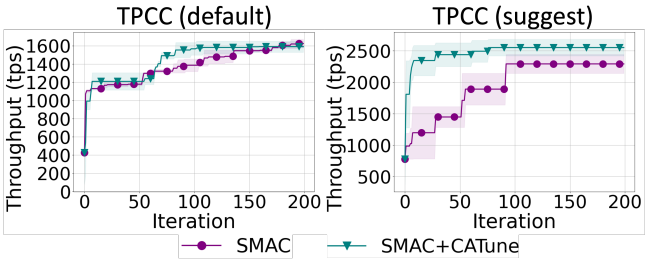

Figure 6: Best performance achieved by CATune under manual default ranges and knowledge-guided ranges on TPC-C on MySQL.

8.3 Generalizing across Optimizers

Scenarios. To demonstrate that CATune is not tied to a specific optimizer or BO implementation, we evaluate it with different surrogate models and optimization frameworks under TPC-C workload. In addition to SMAC (random forest surrogate), we consider Gaussian Process-based BO (GP-BO) and the coarse-to-fine BO framework used in GPTuner. We follow the same experimental protocol as in Section 8.2, optimizing for final performance and measuring sample efficiency. The evaluated scenarios are:

- *GPBO:* GP-based BO with knowledge-guided suggested ranges.
- *GPBO+CATune:* GP-based BO with suggested ranges combined with CATune.
- *GPTuner:* Coarse-to-fine BO with knowledge-guided ranges.
- *GPTuner+CATune:* GPTuner augmented with CATune.

Results: GP-BO. Figure 7 shows results under knowledge-guided reduced ranges. The improvements remain consistent. CATune reaches the baseline GP-BO optimum 8.33× faster on TPC-C. In terms of final performance, CATune improves throughput by 28.71% on TPC-C compared to baseline GP-BO.

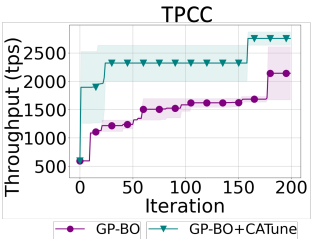

Figure 7: Performance of CATune integrated with GP-BO under knowledge-guided suggested ranges.

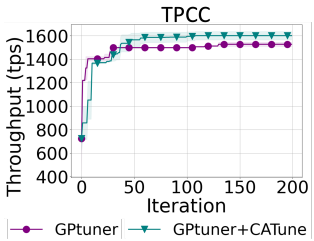

Figure 8: Performance impact of CATune when integrated with coarse-to-fine BO framework from GPTuner.

Results: GPTuner. We next evaluate CATune when coupled with GPTuner's coarse-to-fine BO framework. The same trend holds: CATune accelerates convergence and improves final performance. CATune reaches the baseline GPTuner optimum 5.26× faster and improves throughput by 4.81% on TPC-C as Figure 8 shown.

Summary. Across different surrogate models (random forest vs. Gaussian process) and BO frameworks (standard GP-BO vs. coarse-to-fine GPTuner), CATune consistently improves both sample efficiency and final performance. These results indicate that constraint-aware search-space restriction is orthogonal to the underlying optimizer and generalizes across diverse BO implementations.

8.4 Ablation Study

Scenarios. To isolate the contribution of each component in CATune, we compare two variants with the baseline:

- *SMAC (No rules, baseline):* The original SMAC optimizer operating over the unconstrained configuration space without any explicit constraint handling.
- *SMAC + Rules (Rejection):* A hard constraint enforcement baseline. Configurations violating dependency constraints are rejected and resampled until a valid configuration is obtained. This represents a conventional post hoc constraint-handling strategy.
- *SMAC + Rules + Topo (CATune):* The full CATune design. In addition to rule-based constraints, topology-aware sampling proactively generates configurations that satisfy dependency relationships during the sampling process, thereby avoiding infeasible regions.

Results. Figure 9 compares the three approaches. We make two observations.

First, adding rule-based constraint enforcement substantially improves tuning effectiveness over the unconstrained SMAC baseline. By preventing invalid configurations from being evaluated, SMAC + Rules allocates a larger fraction of the search budget to feasible regions and achieves better tuning performance.

Second, topology-aware sampling further improves both final performance and convergence speed over the rejection-based strategy. While rejection sampling guarantees validity, it repeatedly generates and discards infeasible configurations, resulting in wasted search effort when the feasible region occupies only a small portion of the configuration space. In contrast, topology-aware sampling directly samples from the feasible region by respecting dependency order during configuration generation.

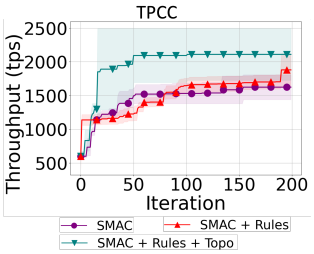

Figure 9: The result of ablation study for CATune, studying different constraint handling strategies.

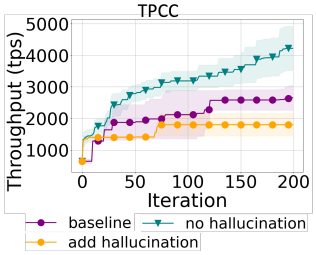

Figure 10: Performance impact between before and after adding constraints that reduce efficiency.

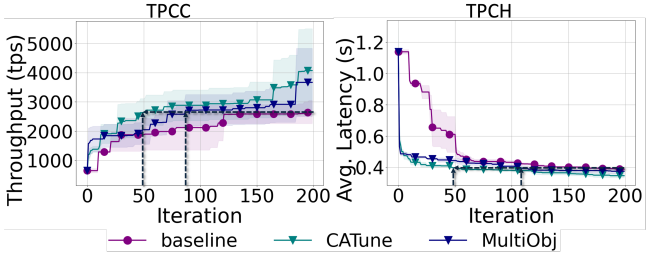

Figure 11: Effect of different employment strategies: proactive filtering (CATune) v.s passive penalization (MultiObj).

As shown in Figure 9, the full system achieves 18.8% higher throughput than SMAC + Rules and reaches the same performance level 2.19× faster. These results indicate that topology-aware sampling not only improves final tuning quality but also substantially increases sample efficiency.

Summary. This ablation study demonstrates two key findings. First, explicit constraint enforcement is necessary for constrained DBMS tuning, as evidenced by the improvement of SMAC + Rules over unconstrained SMAC. Second, topology-aware sampling provides significant additional benefits beyond hard constraint enforcement alone. By proactively incorporating structural dependencies into the sampling process, topology-aware sampling avoids wasted evaluations, improves sample efficiency, and accelerates convergence, making it a critical component of CATune.

9 DISCUSSION

This section analyzes how ordering constraints influence system stability and high-performing configurations, and we compare proactive filtering against penalty-based constraint handling.

9.1 Operational Impact of Constraints

Ordering constraints influence tuning in two distinct ways. First, they prevent configurations that would cause system failures. Second, they reshape the feasible region in ways that may either improve or hinder optimization. We examine both effects empirically.

We begin with correctness-critical constraints whose violation prevents DBMS startup. As a representative example, PostgreSQL requires `superuser_reserved_connections < max_connections`. We conduct tuning over 10 knobs, including these parameters, using LLM-suggested ranges. Across 200 iterations, 9 sampled configurations failed to start the DBMS, and all failures corresponded to violations of this ordering constraint. This result demonstrates that correctness-critical ordering constraints are essential for reliability: without proactive enforcement, the optimizer wastes evaluations on configurations that cannot even initialize the system.

We next examine the opposite scenario—constraints that unnecessarily restrict the search space. To study this effect, we inject two hallucinated ordering constraints produced by a naive LLM-based extraction method. We compare three settings: baseline SMAC with suggested ranges, CATune with valid constraints, and CATune augmented with the two hallucinated constraints.

Figure 10 shows that hallucinated constraints severely degrade tuning performance. Without hallucinations, CATune reaches the baseline optimum 4.55× faster and achieves a 59.15% improvement in final performance. When the two incorrect constraints are injected, the achievable performance improvement drops by 32.07% relative to the baseline. Compared to the hallucinated setting, CATune without hallucinations achieves 134.29% higher final performance and converges 9.09× faster.

These findings highlight a fundamental trade-off. Injecting semantically valid constraints improves reliability and sample efficiency by removing invalid regions while preserving high-performing configurations. In contrast, hallucinated or overly restrictive constraints artificially truncate the feasible domain and may exclude near-optimal configurations. This underscores the importance of high-precision constraint extraction and validation when integrating automatically generated constraints into constraint-aware tuning systems.

9.2 Impact of Different Constraint Employment Strategies

CATune enforces ordering constraints by proactively restricting exploration and exploitation to the feasible region. An alternative approach is to retain the full search space and penalize constraint violations during optimization. We compare these two strategies:

- *Proactive Filtering (CATune):* Infeasible regions are removed from the search space before sampling.
- *Passive Penalization (MultiObj):* The optimizer searches the full domain while minimizing performance loss and constraint violations simultaneously.

For the passive strategy, we formulate tuning as a multi-objective problem. For an ordering constraint $\theta_i \le \theta_j$, the violation penalty is defined as:

$$\mathcal{P}_{ij}(\theta) = \frac{\max(0,\ \theta_i - \theta_j)}{|\theta_j| + 1}. \tag{11}$$

The total penalty across all ordering constraints is:

$$\mathcal{P}(\theta) = \sum_{(i,op,j)\in C} \mathcal{P}_{ij}(\theta), \tag{12}$$

where $op \in \{<, \le\}$. The denominator normalizes violations to reduce scale sensitivity across knobs with heterogeneous ranges. We implement this strategy using ParEGo [21] within the SMAC framework.

Example 4. *Consider an ordering constraint* `superuser_reserved_connections` ≤ `max_connections` *with* `superuser_reserved_connections` = 120 *and* `max_connections` = 100. *The violation magnitude is* 120 − 100 = 20, *and the normalized penalty becomes* $\mathcal{P}_{ij}(\theta) = \frac{20}{|100|+1} \approx 0.198$. *If* `superuser_reserved_connections`= 4, `max_connections`= 6, *the penalty is zero.*

Results. Figure 11 shows the convergence curves on TPC-C and TPC-H. On average, CATune reaches the baseline optimum approximately ∼ 3.93× and 3.57× faster than the baseline on TPC-C and TPC-H, respectively, while MultiObj achieves only 2.3× and 1.74× speedups. This indicates that proactively restricting the search space leads to substantially faster convergence.

After 200 iterations, CATune improves average throughput on TPC-C by 53.52% over the baseline, compared to 38.26% achieved by MultiObj. On TPC-H, CATune reduces latency by 10.73%, whereas MultiObj achieves a 3.43% reduction. Overall, CATune consistently delivers larger performance improvements across both workloads.
Discussion. The weaker performance of passive penalization arises from its need to explore infeasible configurations within the full search space. Although violations are penalized, evaluations are still consumed by invalid or low-quality regions. In contrast, proactive filtering removes infeasible regions entirely, concentrating the optimization process on semantically valid configurations.

These results demonstrate that structural constraint injection is more sample-efficient and effective than penalty-based handling.

9.3 Efficiency of Different Sampling Strategies

To isolate the sampling efficiency of topology-aware sampling, we compare it with three simpler constraint-handling strategies on PostgreSQL using tuning settings (Section 8.1) with the default knob ranges:

- *No Rules*: configurations are sampled from the original search space without constraint handling.
- *Reject*: configurations are repeatedly sampled from the original search space until they satisfy all constraints.
- *Repair*: configurations are first sampled from the original search space and then modified to satisfy violated constraints.
- *Topology-aware Sampling* (Topo): constraints are incorporated before sampling, so generated configurations are feasible by construction.

Results. Table 5 shows that unconstrained sampling is highly inefficient: 99 out of 100 configurations violate at least one constraint. Rejection sampling avoids invalid outputs, but it must generate 15,992 candidates to obtain 100 feasible ones, rejecting 15,892 samples and taking 6.841 seconds. Repair also starts from the unconstrained space; since 99 out of 100 samples require repair, it introduces an extra post-processing step before configurations can be used. In contrast, topology-aware sampling generates feasible configurations directly, returning 100 valid samples from exactly 100 generated candidates without rejection or repair. This avoids both wasted trials and post-hoc modification, making topology-aware sampling more efficient for constrained tuning.

Table 5: Sampling efficiency under PostgreSQL tuning settings with default knob ranges over 100 requested samples (Returned). Generated denotes the actual number of generated configurations, Invalid counts configurations that violate at least one rule, and Time (s) reports the time to return 100 valid configurations. * indicates the number of repaired configurations.

Method	Returned	Generated	Invalid	Time (s)
No Rules	100	100	99	0.026
Reject	100	15,992	15,892	6.841
Repair	100	100	99*	0.031
Topo	100	100	0	0.089

10 RELATED WORK

This section reviews prior work on database configuration tuning and constrained optimization, and situates our approach within the existing literature.

10.1 Database Configuration Tuning

Automated database configuration tuning has been studied extensively and can be broadly categorized into heuristic-based, Bayesian Optimization (BO)-based [43], and Reinforcement Learning (RL)-based approaches [15].

Rule-based methods rely on manually crafted tuning rules to guide exploration [10, 23]. Search-based methods employ heuristic search strategies (e.g., avoiding revisits and exploring local neighborhoods) to iteratively refine configurations [3, 61]. These approaches become less effective as the configuration space grows in dimensionality and exhibits complex inter-knob dependencies.

BO-based methods [6, 12, 22, 47, 56, 57] treat tuning as a black-box optimization problem. They iteratively fit a probabilistic surrogate model that maps knob configurations to performance and select new configurations by maximizing an acquisition function. These methods improve sample efficiency but typically assume box-constrained search spaces and do not explicitly model structural dependencies among knobs.

RL-based approaches explore the configuration space through sequential decision-making, balancing exploration and exploitation via agent–environment interaction [5, 28, 46, 49, 54]. While effective in dynamic settings, RL methods similarly operate over nominal configuration domains without enforcing structural validity constraints by construction.

10.2 Constrained Bayesian Optimization

Constrained Bayesian Optimization (CBO) extends BO to handle unknown or partially observed constraints [13, 14, 36, 40]. In classical CBO, constraints are modeled as black-box functions with probabilistic surrogates, and feasibility is incorporated through acquisition strategies such as Expected Improvement with Constraints or Probability of Feasibility. These methods generally require sampling infeasible configurations to learn constraint boundaries.

In database tuning, ResTune [56] applies constrained optimization to enforce service-level objectives (e.g., throughput or latency bounds). Such constraints are workload-dependent and evaluated

after executing candidate configurations. Feasibility is therefore determined post hoc through runtime measurements.

Our setting differs fundamentally. We focus on deterministic knob dependency constraints that are intrinsic to the configuration space (e.g., ordering constraints among knobs). Such constraints are static, system-defined, and independent of workload behavior. Instead of modeling feasibility probabilistically or verifying it after execution, we encode structural dependencies directly into the configuration domain so that all sampled configurations are valid by construction. This eliminates infeasible trials *a priori*, which is particularly important in database systems where dependency violations may cause instability or startup failures [35, 48].

From an optimization perspective, our approach encodes structural constraints directly into the feasible region, restricting optimization to a constraint-consistent subspace. This differs from GPTuner [25], which leverages domain knowledge to refine knob ranges and prioritize important parameters. While GPTuner biases exploration toward promising regions, it does not enforce structural validity among knobs. Our method instead guarantees feasibility at the configuration-space level. In addition, in contrast to prior database tuning systems that treat the configuration space as box-constrained, our approach explicitly encodes deterministic ordering constraints as structural components of the search domain.

11 CONCLUSION

This work reframes DBMS tuning as a problem of structural search-space design. Rather than treating the configuration domain as box-constrained and discovering invalid regions through costly trial-and-error, we demonstrate that deterministic knob dependencies define a structurally consistent subspace that can be enforced prior to optimization. Hence, we proposed CATune, a constraint-aware tuning framework that embeds ordering constraints directly into Bayesian optimization. By enforcing structural consistency during both exploration and exploitation, and by introducing topology-aware sampling, CATune eliminates infeasible regions by construction and avoids the inefficiencies of rejection-based strategies.

More broadly, our results show that explicitly modeling intrinsic configuration dependencies improves both reliability and convergence efficiency. Incorporating structural knowledge into the search domain provides a principled alternative to purely black-box tuning and opens new opportunities for constraint-aware optimization in complex system configuration tasks.

In future work, we plan to extend this framework to support richer classes of structural constraints and to further investigate their interaction with advanced optimization strategies.

REFERENCES

[1] Ashish Agarwal, Clara Wong-Fannjiang, David Sussillo, Katherine Lee, and Orhan Firat. 2018. Hallucinations in Neural Machine Translation.

[2] Sebastian Ament, Samuel Daulton, David Eriksson, Maximilian Balandat, and Eytan Bakshy. 2023. Unexpected improvements to expected improvement for Bayesian optimization. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 904, 36 pages.

[3] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: an extensible framework for program autotuning. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation* (Edmonton, AB, Canada) *(PACT '14)*. Association for Computing Machinery, New York, NY, USA, 303–316. https://doi.org/10.1145/2628071.2628092

[4] Roger Arnau, José M. Calabuig, Luis M. García-Raffi, Enrique A. Sánchez Pérez, and Sergi Sanjuan. 2024. A Bellman–Ford Algorithm for the Path-Length-Weighted Distance in Graphs. *Mathematics* 12, 16 (2024). https://doi.org/10.3390/math12162590

[5] Baoqing Cai, Yu Liu, Ce Zhang, Guangyu Zhang, Ke Zhou, Li Liu, Chunhua Li, Bin Cheng, Jie Yang, and Jiashu Xing. 2022. HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 646–659. https://doi.org/10.1145/3514221.3517882

[6] Stefano Cereda, Stefano Valladares, Paolo Cremonesi, and Stefano Doni. 2021. CGPTuner: a contextual gaussian process bandit approach for the automatic tuning of IT configurations under varying workload conditions. *Proc. VLDB Endow.* 14, 8 (April 2021), 1401–1413. https://doi.org/10.14778/3457390.3457404

[7] Bertrand Charpentier, Simon Kibler, and Stephan Günnemann. 2022. Differentiable DAG Sampling. arXiv:2203.08509 [cs.LG] https://arxiv.org/abs/2203.08509

[8] ClickHouse, Inc. 2026. ClickHouse Documentation: Server Settings. Online documentation. https://clickhouse.com/docs/operations/server-configuration-parameters/settings Accessed: 2026-06-21.

[9] ClickHouse, Inc. 2026. ClickHouse Documentation: Session Settings. Online documentation. https://clickhouse.com/docs/operations/settings/settings Accessed: 2026-06-21.

[10] Benoît Dageville and Mohamed Zait. 2002. SQL memory management in Oracle9i. In *Proceedings of the 28th International Conference on Very Large Data Bases* (Hong Kong, China) *(VLDB '02)*. VLDB Endowment, 962–973.

[11] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. 2013. OLTP-Bench: an extensible testbed for benchmarking relational databases. *Proc. VLDB Endow.* 7, 4 (Dec. 2013), 277–288. https://doi.org/10.14778/2732240.2732246

[12] Songyun Duan, Vamsidhar Thummala, and Shivnath Babu. 2009. Tuning database configuration parameters with iTuned. *Proc. VLDB Endow.* 2, 1 (Aug. 2009), 1246–1257. https://doi.org/10.14778/1687627.1687767

[13] Jacob R. Gardner, Matt J. Kusner, Zhixiang Xu, Kilian Q. Weinberger, and John P. Cunningham. 2014. Bayesian optimization with inequality constraints. In *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32* (Beijing, China) *(ICML'14)*. JMLR.org, II–937–II–945.

[14] Michael A. Gelbart, Jasper Snoek, and Ryan P. Adams. 2014. Bayesian optimization with unknown constraints. In *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence* (Quebec City, Quebec, Canada) *(UAI'14)*. AUAI Press, Arlington, Virginia, USA, 250–259.

[15] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) *(AAAI'18/IAAI'18/EAAI'18)*. AAAI Press, Article 392, 8 pages.

[16] Lei Huang, Xiaocheng Feng, Weitao Ma, Yuxuan Gu, Weihong Zhong, Xiachong Feng, Weijiang Yu, Weihua Peng, Duyu Tang, Dandan Tu, and Bing Qin. 2024. Learning Fine-Grained Grounded Citations for Attributed Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 14095–14113. https://doi.org/10.18653/v1/2024.findings-acl.838

[17] Frank Hutter and Steve Ramage. 2015. *Manual for SMAC Version v2.10.03-master*. Department of Computer Science, University of British Columbia. https://www.cs.ubc.ca/labs/algorithms/Projects/SMAC/v2.10.03/manual.pdf Version v2.10.03-master.

[18] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards Mitigating LLM Hallucination via Self Reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1827–1843. https://doi.org/10.18653/v1/2023.findings-emnlp.123

[19] Konstantinos Kanellis, Ramnatthan Alagappan, and Shivaram Venkataraman. 2020. Too many knobs to tune? towards faster database tuning by pre-selecting important knobs. In *Proceedings of the 12th USENIX Conference on Hot Topics in Storage and File Systems (HotStorage '20)*. USENIX Association, USA, Article 16, 1 pages.

[20] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: sample-efficient DBMS configuration tuning. *Proc. VLDB Endow.* 15, 11 (July 2022), 2953–2965. https://doi.org/10.14778/3551793.3551844

[21] J. Knowles. 2006. ParEGO: a hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE Transactions on Evolutionary Computation* 10, 1 (2006), 50–66. https://doi.org/10.1109/TEVC.2005.851274

[22] Mayuresh Kunjir and Shivnath Babu. 2020. Black or White? How to Develop an AutoTuner for Memory-based Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD*

'20). Association for Computing Machinery, New York, NY, USA, 1667–1683. https://doi.org/10.1145/3318464.3380591

[23] Eva Kwan. 2002. Automatic Configuration for IBM ® DB 2 Universal Database TM Compressing years of performance tuning experience into seconds of execution. https://api.semanticscholar.org/CorpusID:15267980

[24] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proc. VLDB Endow.* 17, 8 (April 2024), 1939–1952. https://doi.org/10.14778/3659437.3659449

[25] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2025. GPTuner: An LLM-Based Database Tuning System. *SIGMOD Rec.* 54, 1 (April 2025), 101–110. https://doi.org/10.1145/3733620.3733641

[26] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Yuanchun Zhou, Mingjie Tang, and Jianguo Wang. 2024. A Demonstration of GPTuner: A GPT-Based Manual-Reading Database Tuning System. In *Companion of the 2024 International Conference on Management of Data* (<conf-loc>, <city>Santiago AA</city>, <country>Chile</country>, </conf-loc>) *(SIGMOD/PODS '24)*. Association for Computing Machinery, New York, NY, USA, 504–507. https://doi.org/10.1145/3626246.3654739

[27] Cheng Li, Santu Rana, Sunil Gupta, Vu Nguyen, Svetha Venkatesh, Alessandra Sutti, David Rubin, Teo Slezak, Murray Height, Mazher Mohammed, and Ian Gibson. 2019. Accelerating Experimental Design by Incorporating Experimenter Hunches. arXiv:1907.09065 [stat.ML] https://arxiv.org/abs/1907.09065

[28] Guoliang Li, Xuanhe Zhou, Shifu Li, and Bo Gao. 2019. QTune: a query-aware database tuning system with deep reinforcement learning. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2118–2130. https://doi.org/10.14778/3352063.3352129

[29] Yinghao Li, Rampi Ramprasad, and Chao Zhang. 2024. A simple but effective approach to improve structured language model output for information extraction. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 5133–5148.

[30] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, André Biedenkapp, Difan Deng, Carolin Benjamins, Tim Ruhkopf, René Sass, and Frank Hutter. 2022. SMAC3: a versatile Bayesian optimization package for hyperparameter optimization. *J. Mach. Learn. Res.* 23, 1, Article 54 (Jan. 2022), 9 pages.

[31] Seth McCammon, Dylan Jones, and Geoffrey A. Hollinger. 2020. Topology-Aware Self-Organizing Maps for Robotic Information Gathering. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1717–1724. https://doi.org/10.1109/IROS45743.2020.9341040

[32] Oracle Corporation. 2026. MySQL 8.0 Reference Manual: Server System Variables. Online documentation. https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html Accessed: 2026-06-21.

[33] Oracle Corporation. 2026. MySQL 8.0 Reference Manual: System Variable Index. Online documentation. https://dev.mysql.com/doc/refman/8.0/en/dynindex-sysvar.html Accessed: 2026-06-21.

[34] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Conference on Innovative Data Systems Research*. https://api.semanticscholar.org/CorpusID:265531108

[35] Andrew Pavlo, Matthew Butrovich, Lin Ma, Prashanth Menon, Wan Shen Lim, Dana Van Aken, and William Zhang. 2021. Make your database system dream of electric sheep: towards self-driving operation. *Proc. VLDB Endow.* 14, 12 (July 2021), 3211–3221. https://doi.org/10.14778/3476311.3476411

[36] Victor Picheny. 2015. Multiobjective optimization using Gaussian process emulators via stepwise uncertainty reduction. *Statistics and Computing* 25, 6 (Nov. 2015), 1265–1280. https://doi.org/10.1007/s11222-014-9477-x

[37] PostgreSQL Global Development Group. 2026. PostgreSQL 13 Documentation: Resource Consumption. Online documentation. https://www.postgresql.org/docs/13/runtime-config-resource.html Accessed: 2026-06-21.

[38] PostgreSQL Global Development Group. 2026. PostgreSQL 13 Documentation: Server Configuration. Online documentation. https://www.postgresql.org/docs/13/runtime-config.html Accessed: 2026-06-21.

[39] PostgreSQL Global Development Group. 2026. PostgreSQL 13 Documentation: Write Ahead Log. Online documentation. https://www.postgresql.org/docs/13/runtime-config-wal.html Accessed: 2026-06-21.

[40] Matthias Schonlau, William Welch, and Donald Jones. 1998. *Global versus local search in constrained optimization of computer models*. Vol. 34. 11–25. https://doi.org/10.1214/lnms/1215456182

[41] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI] https://arxiv.org/abs/2303.11366

[42] Alan Skelley. 2000. DB2 Advisor: An Optimizer Smart Enough to Recommend its own Indexes. In *Proceedings of the 16th International Conference on Data Engineering (ICDE '00)*. IEEE Computer Society, USA, 101.

[43] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*, F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf

[44] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that "Reads the Manual". In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 190–203. https://doi.org/10.1145/3514221.3517843

[45] Immanuel Trummer. 2022. Demonstrating DB-BERT: A Database Tuning Tool that "Reads" the Manual. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2437–2440. https://doi.org/10.1145/3514221.3520171

[46] Immanuel Trummer, Junxiong Wang, Ziyun Wei, Deepak Maram, Samuel Moseley, Saehan Jo, Joseph Antonakakis, and Ankush Rayabhari. 2021. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning. *ACM Trans. Database Syst.* 46, 3, Article 9 (Sept. 2021), 45 pages. https://doi.org/10.1145/3464389

[47] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024. https://doi.org/10.1145/3035918.3064029

[48] Dana Van Aken, Dongsheng Yang, Sebastien Brillard, Ari Fiorino, Bohan Zhang, Christian Bilien, and Andrew Pavlo. 2021. An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems. *Proc. VLDB Endow.* 14, 7 (March 2021), 1241–1253. https://doi.org/10.14778/3450980.3450992

[49] Junxiong Wang, Immanuel Trummer, and Debabrota Basu. 2021. UDO: universal database optimization using reinforcement learning. *Proc. VLDB Endow.* 14, 13 (Sept. 2021), 3402–3414. https://doi.org/10.14778/3484224.3484236

[50] Tevin Wang and Chenyan Xiong. 2025. AutoRule: Reasoning Chain-of-thought Extracted Rule-based Rewards Improve Preference Learning. arXiv:2506.15651 [cs.LG] https://arxiv.org/abs/2506.15651

[51] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, Chaoya Jiang, Rui Xie, Jindong Wang, Xing Xie, Wei Ye, Shikun Zhang, and Yue Zhang. 2024. PandaLM: An Automatic Evaluation Benchmark for LLM Instruction Tuning Optimization. arXiv:2306.05087 [cs.CL] https://arxiv.org/abs/2306.05087

[52] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2025. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL] https://arxiv.org/abs/2401.11817

[53] Dani Yogatama and Gideon S. Mann. 2014. Efficient Transfer Learning Method for Automatic Hyperparameter Tuning. In *International Conference on Artificial Intelligence and Statistics*. https://api.semanticscholar.org/CorpusID:319311

[54] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 415–432. https://doi.org/10.1145/3299869.3300085

[55] Xinyi Zhang, Zhuo Chang, Yang Li, Hong Wu, Jian Tan, Feifei Li, and Bin Cui. 2022. Facilitating database tuning with hyper-parameter optimization: a comprehensive experimental evaluation. *Proc. VLDB Endow.* 15, 9 (May 2022), 1808–1821. https://doi.org/10.14778/3538598.3538604

[56] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuowei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. ResTune: Resource Oriented Tuning Boosted by Meta-Learning for Cloud Databases. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2102–2114. https://doi.org/10.1145/3448016.3457291

[57] Xinyi Zhang, Hong Wu, Yang Li, Jian Tan, Feifei Li, and Bin Cui. 2022. Towards Dynamic and Safe Configuration Tuning for Cloud Databases. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 631–645. https://doi.org/10.1145/3514221.3526176

[58] Xinyi Zhang, Hong Wu, Yang Li, Zhengju Tang, Jian Tan, Feifei Li, and Bin Cui. 2023. An Efficient Transfer Learning Based Configuration Adviser for Database Tuning. *Proc. VLDB Endow.* 17, 3 (Nov. 2023), 539–552. https://doi.org/10.14778/3632093.3632114

[59] Yudi Zhang, Pei Xiao, Lu Wang, Chaoyun Zhang, Meng Fang, Yali Du, Yevgeniy Puzyrev, Randolph Yao, Si Qin, Qingwei Lin, Mykola Pechenizkiy, Dongmei Zhang, Saravan Rajmohan, and Qi Zhang. 2024. RuAG: Learned-rule-augmented Generation for Large Language Models. arXiv:2411.03349 [cs.AI] https://arxiv.org/abs/2411.03349

[60] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang,

Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2020, 29 pages.

[61] Yuqing Zhu, Jianxun Liu, Mengying Guo, Yungang Bao, Wenlong Ma, Zhuoyue Liu, Kunpeng Song, and Yingchun Yang. 2017. BestConfig: tapping the performance potential of systems via automatic configuration tuning. In *Proceedings of the 2017 Symposium on Cloud Computing* (Santa Clara, California) *(SoCC '17)*. Association for Computing Machinery, New York, NY, USA, 338–350. https://doi.org/10.1145/3127479.3128605